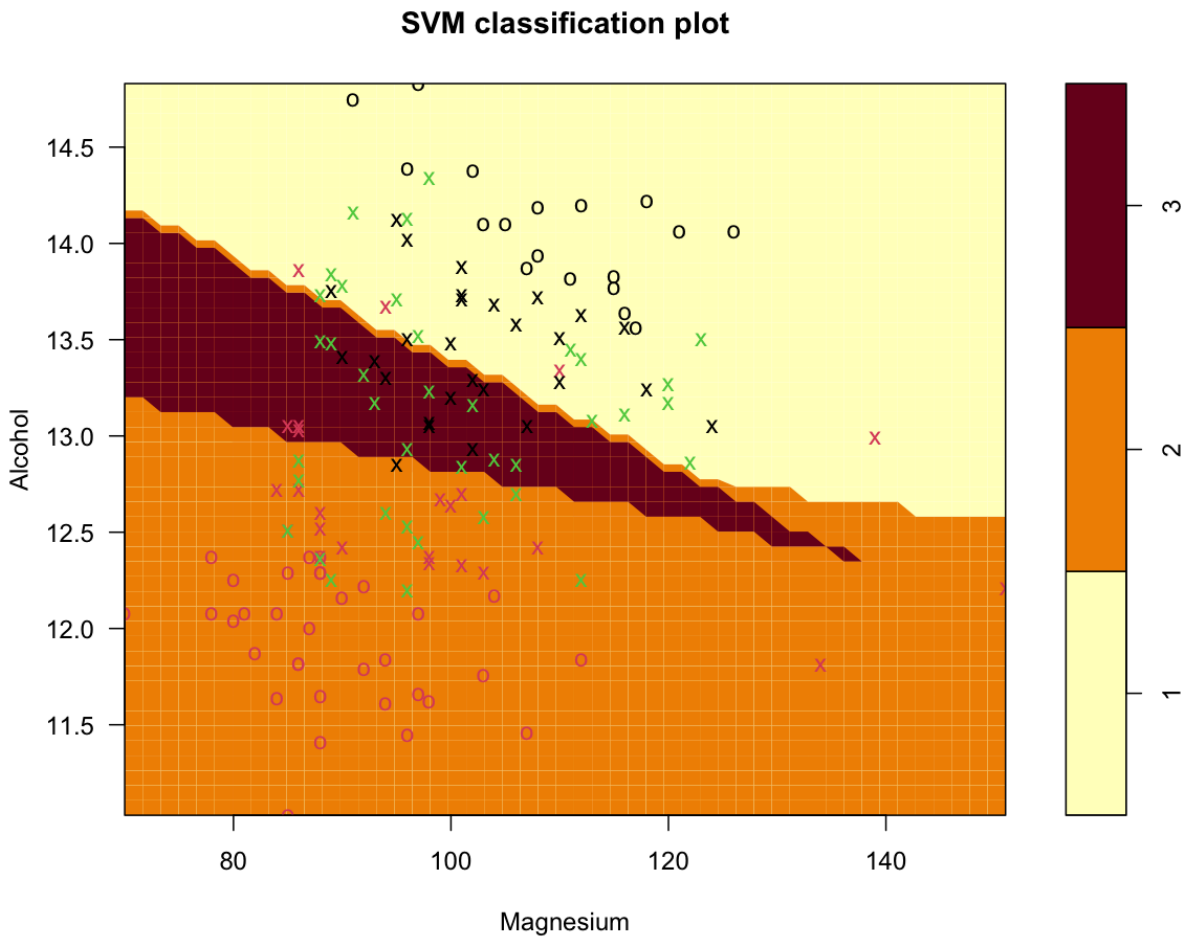
Sean McHugh

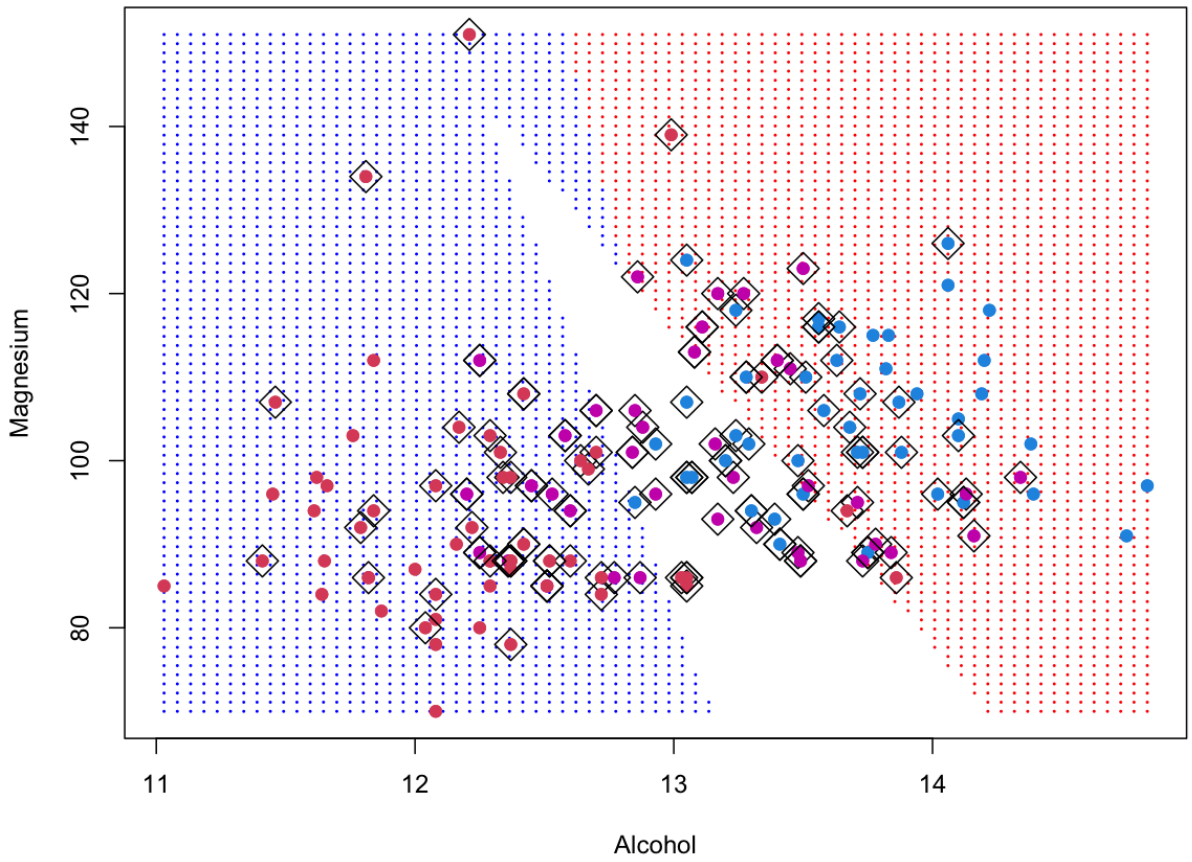
```
#Lab 5
> library(caret)
Loading required package: ggplot2
Loading required package: lattice
> library(e1071)

Attaching package: ‘e1071’

The following object is masked from ‘package:ggplot2’:

    element
> library(GGally)
> library(class)
> #Read dataset
> dataset <- read.csv("~/Downloads/wine.data")
> names(dataset) <- c("Type","Alcohol","Malic acid","Ash","Alcalinity of ash","Magnesium","Total
phenols","Flavanoids","Nonflavanoid Phenols","Proanthocyanins","Color
Intensity","Hue","Od280/od315 of diluted wines","Proline")
> dataset$Type <- as.factor(dataset$Type)
> View(dataset)
> #Split train/test
> N <- nrow(dataset)
> train.indexes <- sample(N,0.8*N)
> train <- dataset[train.indexes,]
> test <- dataset[-train.indexes,]
> #Separate x (features) & y (Type)
> X <- dataset[,2:14]
> Y <- dataset[,1]
> #SVM training model - linear kernel
> svm.mod0 <- svm(Type ~ Alcohol + Magnesium, data = train, kernel = 'linear')
> svm.mod0

Call:
svm(formula = Type ~ Alcohol + Magnesium, data = train, kernel = "linear")


Parameters:
   SVM-Type:  C-classification
 SVM-Kernel:  linear
       cost:  1

Number of Support Vectors:  90
```

```
> plot(svm.mod0, data = train, formula = Alcohol~Magnesium, svSymbol = "x", dataSymbol =
"o")
```



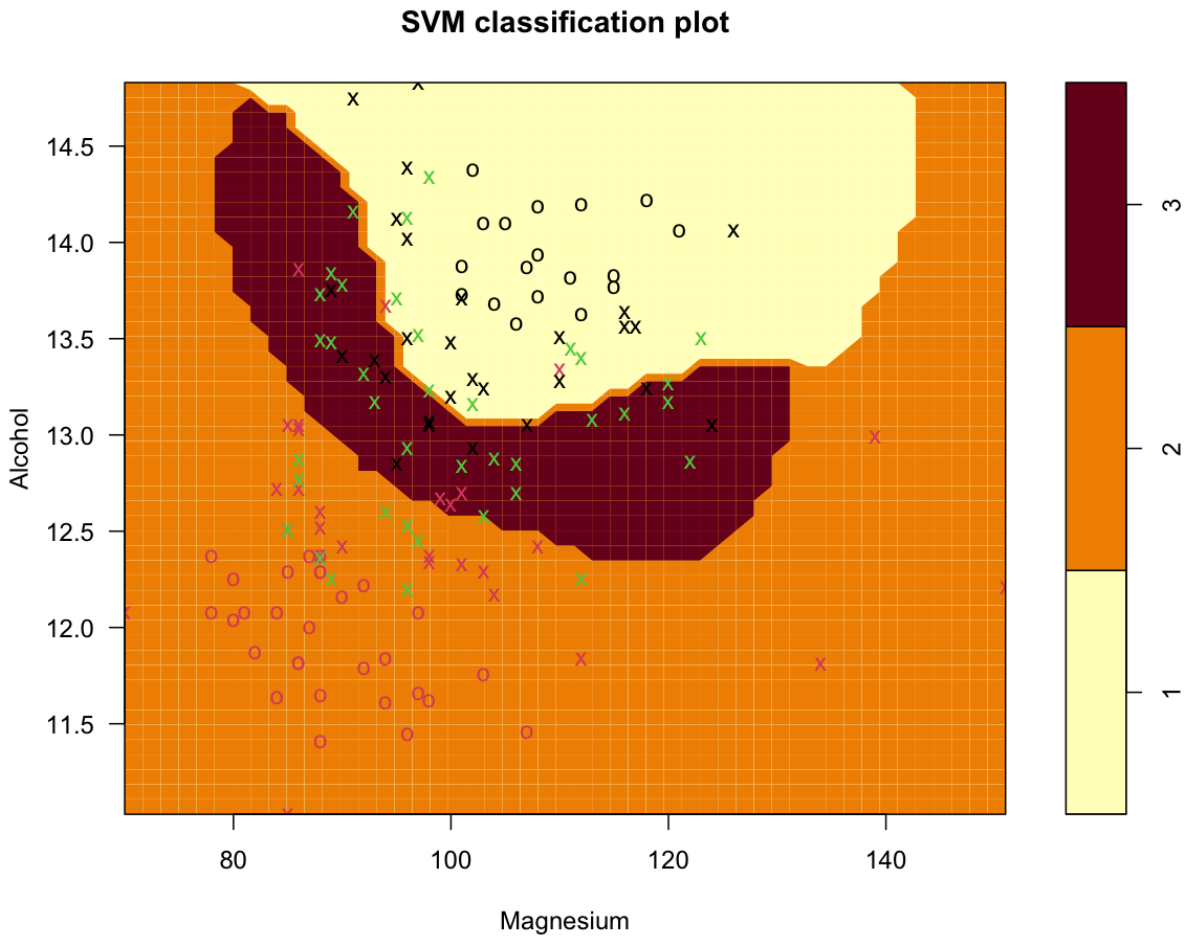

```
> train.pred <- predict(svm.mod0, train)
> cm = as.matrix(table(Actual = train$Type, Predicted = train.pred))
> cm
      Predicted
Actual  1  2  3
     1 34  0 12
     2  4 50  3
     3 16 12 10
> n = sum(cm) # number of instances
> nc = nrow(cm) # number of classes
> diag = diag(cm) # number of correctly classified instances per class
> rowsums = apply(cm, 1, sum) # number of instances per class
> colsums = apply(cm, 2, sum) # number of predictions per class
> p = rowsums / n # distribution of instances over the actual classes
> q = colsums / n # distribution of instances over the predicted
```

```
> accuracy <- sum(diag)/n
> accuracy
[1] 0.6666667
> recall = diag / rowsums
> precision = diag / colsums
> f1 = 2 * precision * recall / (precision + recall)
> #SVM training model - linear kernel precision, recall, f1 scores
> data.frame(precision, recall, f1)
  precision    recall        f1
1 0.6296296 0.7391304 0.6800000
2 0.8064516 0.8771930 0.8403361
3 0.4000000 0.2631579 0.3174603
> make.grid = function(X, n = 75) {
+   grange = apply(X, 2, range)
+   X1 = seq(from = grange[1,1], to = grange[2,1], length = n)
+   X2 = seq(from = grange[1,2], to = grange[2,2], length = n)
+   expand.grid(Alcohol = X1, Magnesium = X2)
+ }
> X <- train[,c(2,6)]
> Y <- as.numeric(train$Type)
> Y[Y==2] <- -1
> xgrid = make.grid(X)
> ygrid = predict(svm.mod0, xgrid)
> plot(xgrid, col = c("red","blue")[as.numeric(ygrid)], pch = 20, cex = .2)
```




```
> points(X, col = Y + 3, pch = 19)
> points(X[svm.mod0$index,], pch = 5, cex = 2)
> #SVM test model - linear kernel
> test.pred <- predict(svm.mod0, test)
> cm = as.matrix(table(Actual = test$Type, Predicted = test.pred))
> cm
      Predicted
Actual  1  2  3
     1 10  0  2
     2  0 12  2
     3  4  1  5
> n = sum(cm) # number of instances
> nc = nrow(cm) # number of classes
> diag = diag(cm) # number of correctly classified instances per class
> rowsums = apply(cm, 1, sum) # number of instances per class
> colsums = apply(cm, 2, sum) # number of predictions per class
> p = rowsums / n # distribution of instances over the actual classes
> q = colsums / n # distribution of instances over the predicted
```

```
> accuracy <- sum(diag)/n
> accuracy
[1] 0.75
> recall = diag / rowsums
> precision = diag / colsums
> f1 = 2 * precision * recall / (precision + recall)
> #SVM test model - linear kernel precision, recall, f1 scores
> data.frame(precision, recall, f1)
  precision    recall        f1
1 0.7142857 0.8333333 0.7692308
2 0.9230769 0.8571429 0.8888889
3 0.5555556 0.5000000 0.5263158
> #Second SVM train model - polynomial kernel
> svm.mod1 <- svm(Type ~ Alcohol+Magnesium, data = train, kernel = 'radial')
> plot(svm.mod1, train, Alcohol~Magnesium)
> train.pred <- predict(svm.mod1, train)
> ygrid = predict(svm.mod1, xgrid)
> plot(xgrid, col = as.numeric(ygrid), pch = 20, cex = .2)
```



```
> points(X, col = Y + 1, pch = 19)
> points(X[svm.mod0$index,], pch = 5, cex = 2)
> points(X, col = Y + 3, pch = 19)
> cm = as.matrix(table(Actual = train$Type, Predicted = train.pred))
> cm
      Predicted
Actual  1  2  3
     1 35  0 11
     2  1 51  5
     3  9 10 19
> n = sum(cm)
> nc = nrow(cm)
> diag = diag(cm)
> rowsums = apply(cm, 1, sum)
> colsums = apply(cm, 2, sum)
> p = rowsums / n
> q = colsums / n
> accuracy <- sum(diag)/n
> accuracy
[1] 0.7446809
> recall = diag / rowsums
> precision = diag / colsums
> f1 = 2 * precision * recall / (precision + recall)
> #Second SVM train model - polynomial kernel precision, recall, f1 scores
> data.frame(precision, recall, f1)
  precision    recall        f1
1 0.7777778 0.7608696 0.7692308
2 0.8360656 0.8947368 0.8644068
3 0.5428571 0.5000000 0.5205479
> #Second SVM testing model - polynomial kernel
> test.pred <- predict(svm.mod1, test)
> cm = as.matrix(table(Actual = test$Type, Predicted = test.pred))
> cm
      Predicted
Actual  1  2  3
     1 11  0  1
     2  0 13  1
     3  3  1  6
> n = sum(cm) # number of instances
> nc = nrow(cm) # number of classes
> diag = diag(cm) # number of correctly classified instances per class
> rowsums = apply(cm, 1, sum) # number of instances per class
> colsums = apply(cm, 2, sum) # number of predictions per class
> p = rowsums / n # distribution of instances over the actual classes
```

```
> q = colsums / n # distribution of instances over the predicted
> accuracy <- sum(diag)/n
> accuracy
[1] 0.8333333
> recall = diag / rowsums
> precision = diag / colsums
> f1 = 2 * precision * recall / (precision + recall)
> #Second SVM testing model - polynomial kernel precision, recall, f1 scores
> data.frame(precision, recall, f1)
  precision    recall        f1
1 0.7857143 0.9166667 0.8461538
2 0.9285714 0.9285714 0.9285714
3 0.7500000 0.6000000 0.6666667
> #Tuned SVM - polynomial kernel to find optimum C and gamma values
> gamma.range <- seq(0.1,10, .1)
> gamma.range
  [1]  0.1  0.2  0.3  0.4  0.5  0.6  0.7  0.8  0.9  1.0  1.1  1.2  1.3  1.4  1.5  1.6  1.7  1.8  1.9  2.0  2.1
2.2  2.3  2.4  2.5  2.6  2.7  2.8  2.9  3.0
 [31]  3.1  3.2  3.3  3.4  3.5  3.6  3.7  3.8  3.9  4.0  4.1  4.2  4.3  4.4  4.5  4.6  4.7  4.8  4.9  5.0
5.1  5.2  5.3  5.4  5.5  5.6  5.7  5.8  5.9  6.0
 [61]  6.1  6.2  6.3  6.4  6.5  6.6  6.7  6.8  6.9  7.0  7.1  7.2  7.3  7.4  7.5  7.6  7.7  7.8  7.9  8.0
8.1  8.2  8.3  8.4  8.5  8.6  8.7  8.8  8.9  9.0
 [91]  9.1  9.2  9.3  9.4  9.5  9.6  9.7  9.8  9.9 10.0
> Cost.range <- seq(1,20, 1)
> Cost.range
 [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
> tuned.svm <- tune.svm(Type~., data = train, kernel = 'polynomial',gamma = gamma.range,
cost = Cost.range)
> tuned.svm

Parameter tuning of ‘svm’:

- sampling method: 10-fold cross validation

- best parameters:
 gamma cost
   0.3    1

- best performance: 0.0352381

> #kNN model
> split.rat <- 0.7
> train.indexes <- sample(150,split.rat*150)
> train <- dataset[train.indexes,]
```

```
> test <- dataset[-train.indexes,]
> mod.knn <- train(Type~Alcohol + Magnesium, data=train, method="knn")
> pred.knn <- predict(mod.knn, newdata = test)
> cm <- table(Actual = test$Type, Predicted = pred.knn)
> cm
      Predicted
Actual  1  2  3
     1 18  1  2
     2  2 17  1
     3 21  6  4
> n = sum(cm)
> nc = nrow(cm)
> diag = diag(cm)
> rowsums = apply(cm, 1, sum)
> colsums = apply(cm, 2, sum)
> p = rowsums / n
> q = colsums / n
> accuracy <- sum(diag)/n
> accuracy
[1] 0.5416667
> recall = diag / rowsums
> precision = diag / colsums
> f1 = 2 * precision * recall / (precision + recall)
> #kNN model precision, recall, f1 scores
> data.frame(precision, recall, f1)
  precision    recall        f1
1 0.4390244 0.8571429 0.5806452
2 0.7083333 0.8500000 0.7727273
3 0.5714286 0.1290323 0.2105263
```